future internet

MDPI

Article

Multi-Class Intrusion Detection Based on Transformer for IoT Networks Using CIC-IoT-2023 Dataset

Shu-Ming Tseng [1,*], Yan-Qi Wang [1] and Yung-Chung Wang [2]

[1] Department of Electronic Engineering, National Taipei University of Technology, Taipei 10608, Taiwan
[2] Department of Electrical Engineering, National Taipei University of Technology, Taipei 10608, Taiwan
* Correspondence: shuming@ntut.edu.tw

Abstract: This study uses deep learning methods to explore the Internet of Things (IoT) network intrusion detection method based on the CIC-IoT-2023 dataset. This dataset contains extensive data on real-life IoT environments. Based on this, this study proposes an effective intrusion detection method. Apply seven deep learning models, including Transformer, to analyze network traffic characteristics and identify abnormal behavior and potential intrusions through binary and multivariate classifications. Compared with other papers, we not only use a Transformer model, but we also consider the model's performance in the multi-class classification. Although the accuracy of the Transformer model used in the binary classification is lower than that of DNN and CNN + LSTM hybrid models, it achieves better results in the multi-class classification. The accuracy of binary classification of our model is 0.74% higher than that of papers that also use Transformer on TON-IOT. In the multi-class classification, our best-performing model combination is Transformer, which reaches 99.40% accuracy. Its accuracy is 3.8%, 0.65%, and 0.29% higher than the 95.60%, 98.75%, and 99.11% figures recorded in papers using the same dataset, respectively.

Keywords: Internet of Things; intrusion detection; deep learning; CIC-IoT-202; transformer

Citation: Tseng, S.-M.; Wang, Y.-Q.; Wang, Y.-C. Multi-Class Intrusion Detection Based on Transformer for IoT Networks Using CIC-IoT-2023 Dataset. *Future Internet* **2024**, *16*, 284. https://doi.org/10.3390/fi16080284

Academic Editors: Olivier Markowitch and Jean-Michel Dricot

Received: 7 June 2024
Revised: 25 July 2024
Accepted: 31 July 2024
Published: 8 August 2024

1. Introduction

In recent years, Internet of Things technology has developed rapidly, and we have entered a highly interconnected smart world. IoT devices have been integrated into various industries, including healthcare, agriculture, transportation, and manufacturing [1]. Experts predict that by 2025, the Internet of Things and its applications will have a huge economic impact, with the annual impact ranging from 3.9 trillion to 11.1 trillion [2]. However, this seamless connection also brings new challenges, one of which is security. The ever-increasing number of IoT devices makes them potential targets for attacks, so protecting these devices from improper access and attacks has become critical. In such an environment with diverse devices, there are bound to be devices that are more vulnerable to attacks. Such devices not only affect the security of the IoT system, but also affect the transmission channels in the system, and even cause a partial or complete failure of the transmission network [3]. With the advancement of artificial intelligence technology, machine learning (ML) and deep learning (DL) have made great progress and are now widely used in various fields such as wireless communications, computer vision, and healthcare systems [4]. Intrusion detection systems based on machine learning and deep learning are widely used in the Internet of Things environment [5].

Abbas et al. [1] used the CIC-IoT-2023 dataset and used DNN-based federated learning to detect the security of IoT devices through binary classification. The result accuracy rate is 99.0%. Wang et al. [6] compared six DL models, including DNN, CNN, RNN, LSTM, CNN + LSTM, and the CNN + RNN hybrid model, with the CSE-CIC-IDS2018 dataset. The results showed that the CNN + LTSM model performed well in both classifications. The results all have the highest accuracy rates, 98.84% and 98.85%, respectively. Ahmed

et al. [7] compared their proposed Transformer architecture with RNN and LSTM with binary classification using the ToN_IoT dataset released in 2020. The results show that the proposed Transformer model performs excellently in terms of accuracy and precision, with an accuracy rate of 87.79%.

References [7,8] mention the time complexity of some of the models in our paper such as RNN, CNN, LSTM, etc. Reference [6] mentions most of the models' time complexity, in the same way as our paper but in a different dataset.

He et al. [9] proposed a transferable and adaptive network intrusion detection system (NIDS) based on deep reinforcement learning. The results reached 99.60% and 95.60% in the binary classification and multi-class classification of CIC-IoT2023, respectively. Jony et al. [10] used LSTM to conduct an experimental evaluation of the multi-class classification in CIC-IoT-2023, and the accuracy of the results reached 98.75%. Jaradat et al. [11] used four different machine learning methods to classify network attacks in CIC-IoT-2023, but they did not mention the classification tasks they used. Among them, Gradient Boost achieved the highest accuracy of 95%. Among the above-mentioned papers, only Abbas et al. [1] dealt with the problem of data imbalance in the dataset. Table 1 summarizes the key points of the above papers. The effectiveness of machine learning-based intrusion detection systems (ML-IDSs) depends largely on the quality of the dataset [12]. In this paper, we use the CIC-IoT-2023 dataset [13] released in 2023 to conduct IDS experiments. CIC-IoT-2023 is a unique and comprehensive collection of information designed specifically for IoT attacks. And we use multiple models, such as DNN, CNN, RNN, LSTM, CNN + LSTM, CNN + RNN, and Transformer, to identify whether the traffic is malicious. Classification tasks cover binary classification and multi-class classification. The main contributions of this study are detailed below.

(1) We use the CIC-IoT-2023 dataset [1,13] used by Abbas et al. This is currently the largest collection of IoT data recorded by real IoT devices. The number of data entries in this dataset reaches 46,686,579 and there are as many as 33 attack types. Among them, most of the examples in this dataset are related to common malicious attacks: DDoS and DoS attacks [14];
(2) We not only use the six DL models used in [6], but also use a Transformer model [15] to handle binary and multi-class classification tasks. Compared with [1,7], we further implement the multi-class classification on our model;
(3) On the ToN_IoT dataset, compared with [7], our Transformer model achieved an accuracy of 88.25%, which is 0.46% higher than the 87.79% of [7];
(4) Compared with [10,11,13], which also use the CIC-IoT-2023 dataset [16,17], the accuracy of our Transformer model in the multi-class classification reaches 99.40% accuracy; when compared with 95.60% [10], 98.75% [11], and 99.11% [13], our results are 3.8%, 0.65%, and 0.29% higher, respectively.

Table 1. Related works/baseline schemes.

Paper	Dataset	Classification	DL Method	Accuracy	Inference Time [1]
[1]	CIC-IoT-2023	Binary	DNN based on Federated Learning	99.00%	
[6]	CIC-IDS-2018	Binary, Multi-class	DNN, RNN, CNN, LSTM, CNN + LSTM, and CNN + RNN	98.85%	Multi-Class: LSTM: 3.451 (ms) CNN + LSTM: 4.31 (ms)
[7]	ToN-IoT	Binary	LSTM, RNN, and Transformer	87.79%	Binary Class LSTM: 27 (s) RNN: 35 (s)

Table 1. *Cont.*

Paper	Dataset	Classification	DL Method	Accuracy	Inference Time [1]
[9]	CIC-IoT-2023	Multi-class	Deep Reinforcement Learning	95.60%	
[10]	CIC-IoT-2023	Multi-class	LTSM	98.75%	
[11]	CIC-IoT-2023	Not Mentioned	Gradient Boost, MLP, Logistic Regression, and KNN	95.00%	
[13]	CIC-IoT-2023	Binary, Multi-class	DNN	99.44%, 99.11%	
[8]	KD999	Multi-class	CNN, Autoencoder, FCN RNN, U-Net, TCN, and TCN + LSTM	97.7%	Multi-Class CNN: 5 (min/epoch) TCN + LSTM: 11 (min/epoch)

[1] Inference time is copy from references.

The second part of this paper is methodology, which describes the dataset and data preprocessing methods in detail. The third part will introduce six neural network models and Transformer models, and the fourth part will show the experimental results. The fifth part is the conclusion of this paper.

2. Methodology

The system architecture diagram of this paper is shown in Figure 1, which is divided into two parts: data preprocessing and training evaluation. Next, we will introduce the details of the system architecture diagram one by one.

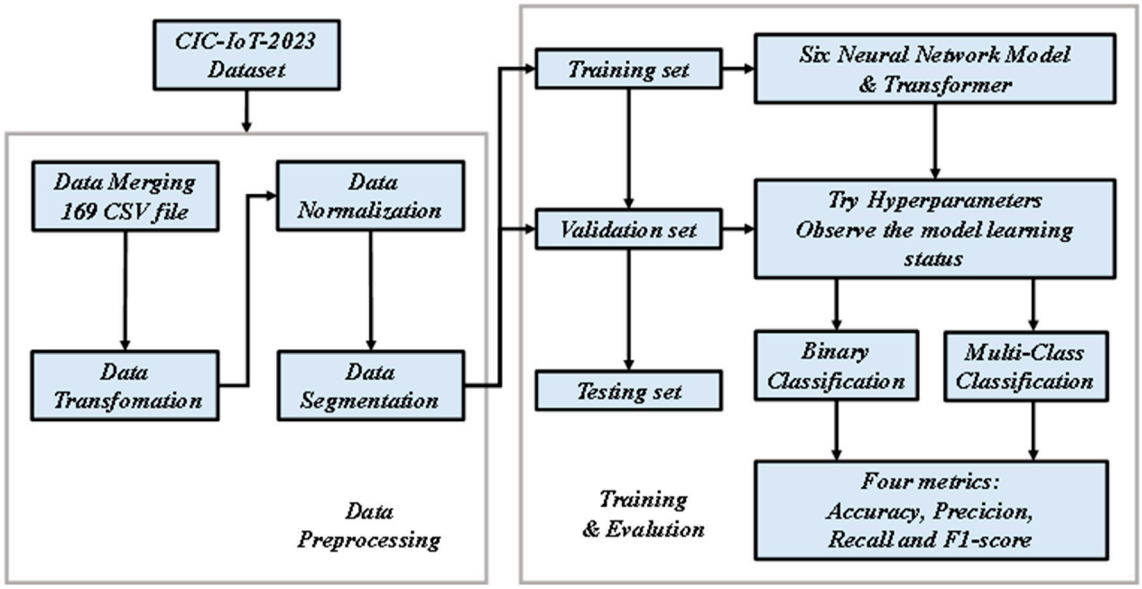


Figure 1. Architecture diagram of this paper.

2.1. CIC-IoT-2023

As of 2023, CIC-IoT-2023 stands out as the largest IoT dataset [16], derived from real IoT devices. The dataset contains data from 105 IoT devices, documenting 33 recorded attacks. Notably, these attacks were launched by malicious IoT devices targeting other IoT devices. In addition, CIC-IoT-2023 also contains multiple attack types that do not exist in other IoT datasets

Table 2 provides the number of each label containing benign traffic. This dataset contains a total of 46 features and 1 label. Different from the 84 features of CSE-CIC-IDS2018, CIC-IoT-2023 has 37 fewer features. In this experiment, no specific feature screening was performed, and all features were used directly to conduct the experiment.

Table 2. The number of each label containing benign traffic.

Label	Quantitys	Label	Quantitys	Label	Quantitys
DDoS-ICMP_Flood	7,200,504	Mirai-greeth_flood	991,866	DoS-HTTP_Flood	71,864
DDoS-UDP_Flood	5,412,287	Mirai-udpplain	890,576	Vulnerability Scan	37,382
DDoS-TCP_Flood	4,497,667	Mirai-greip_flood	751,682	DDoS-SlowLoris	23,246
DDoS-PSHACK_Flood	4,094,755	DDoS-ICMP_Fragmentation	452,489	DictionaryBruteForce	13,064
DDoS-SYN_Flood	4,059,190	MITM-ArpSpoofing	307,593	BrowserHijacking	5859
DDoS-RSTFINFlood	4,045,190	DDoS-UDP_Fragmentation	286,925	CommandInjection	5409
DDoS-SynonymousIP_Flood	3,598,138	DDoS-ACK_Fragmentation	285,104	SQL Injection	5245
DoS-UDP_Flood	3,318,595	Recon-HostDiscovery	178,911	XSS	3946
DoS-TCP_Flood	2,671,445	Recon-OSScan	134,378	Backdoor_Malware	3218
DoS-SYN_Flood	2,028,834	Recon-PortScan	98,259	Recon-PingSweep	2262
Benign	1,098,195	DDoS-HTTP_Flood	71,864	Uploading_Attack	1252

CIC-IoT-2023 Features

CIC-IoT-2023 has 46 features and those features are shown in Table 3.

Table 3. The features used in CIC-IoT-2023.

Feature	Name
1	Flow duration
2	Header Length
3	Protocol
4	Type
5	Duration
6	Rate Mrate Drate
7	fin flag number
8	syn flag number
9	rst flag number
10	psh flag number
11	ack flag number
12	ece flag number
13	cwr flag number
14	ack count
15	syn count
16	fin count
17	urg count
18	rst count
19	HTTP
20	HTTPS
21	DNS
22	Telnet
23	SMTP
24	SSH
25	IRC
26	TCP
27	UDP
28	DHCP
29	ARP
30	ICMP
31	IPv
32	LLC
33	Tot sum
34	Min
35	Max

Table 3. *Cont.*

Feature	Name
36	AVG
37	Std
38	Tot size
39	IAT
40	Number
41	Magnitude
42	Radius
43	Covariance
44	Variance
45	Weight
46	Flow duration

We chose all the above features because all of these features lack redundancy. This method ensures better accuracy.

2.2. Data Merging

Since the dataset is spread across 169 CSV files, it is necessary to merge these files into a single file before importing the data for processing and training. Therefore, as a first step, we will merge all 169 CSV files before proceeding to subsequent stages.

2.3. Data Transformation

In this part, the text labels must be converted to a numeric format so that the model can read the labels In the binary classification, there are two types of labels. The benign label assignment is 0, with a total of 1,098,195 records. The malicious attack label is 1, with a total of 45,588,384 records, making an overall total of 46,686,579 records. In the multi-class classification, we classify malicious attacks into seven categories. Including the benign traffic, there are a total of eight labels [17]. The distribution of converted tags is shown in Figure 2.

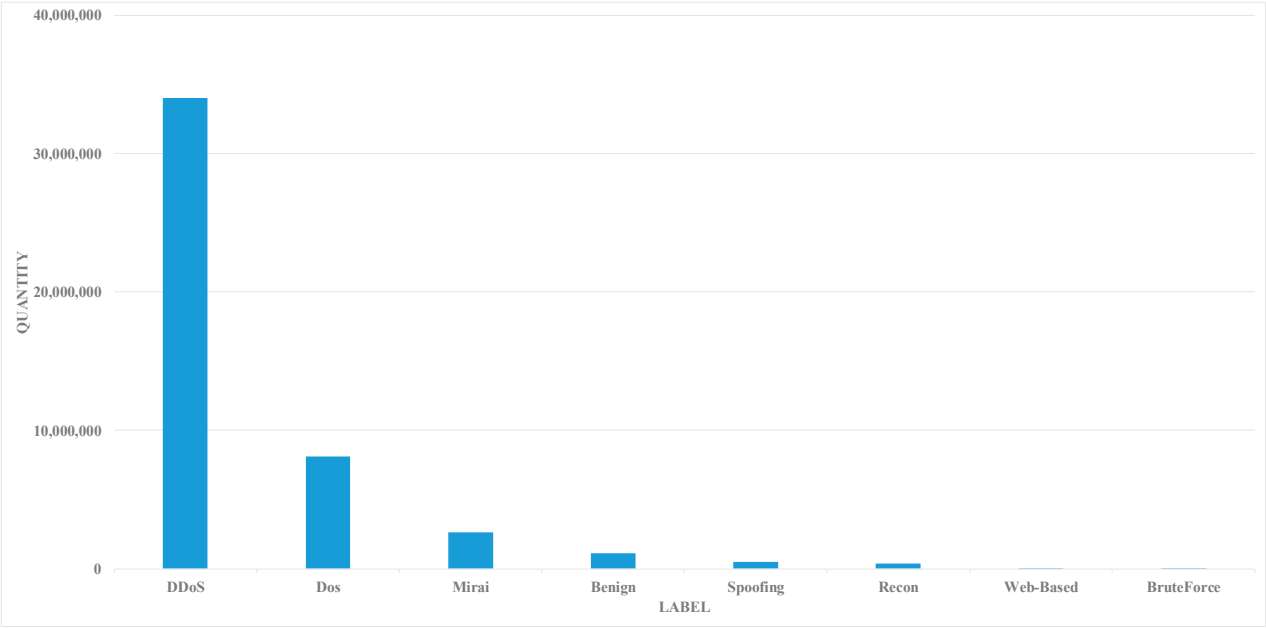


Figure 2. Distribution of converted labels containing benign traffic.

2.4. Data Normalization

In order to improve the performance of deep learning models, feature normalization techniques are usually used to achieve the above purposes. We transform the numerical values of the features so that they are relatively consistent. The method we use is Standard-Scaler technology, which is used to convert the value to a standard normal distribution with a mean of 0 and a standard deviation of 1. This specific method is to calculate the ratio of the difference between the original value and the mean and the standard deviation.

2.5. Data Segmentation

Since the dataset lacks predefined training and testing sets, we used the holdout method for segmentation in this experiment. This technique involves dividing the dataset into a training–validation set and a testing set based on a specified ratio. In this study, we allocate 80% of the dataset to the training–validation set and the remaining 20% to the test set. This partitioning strategy aims to make the model generalizable. Furthermore, in the training–validation set, 80% is designated as the training set, including 37,349,263 records, while the remaining 20% is designated as the validation set, with a total of 9,337,316 records. This distribution corresponds to a proportion of approximately 80% and 20% for the entire dataset [6].

3. Deep Learning Model

In the experiments of this paper, we use the six neural network models mentioned above [6]. In addition to this, we use the Transformer model [7,15] to conduct further experiments. Transformer's self-attention mechanism allows the model to process all positions in the sequence in parallel, unlike RNN, which needs to process them sequentially. This enables Transformer to more effectively utilize computing resources during training and inference and improve the model's training speed. We use brute force to try our best to exhaust various parameter settings to find the best model settings.

3.1. Neural Network

In the neural network, each neural network has six combinations, the hidden layer is set to layer 1 and layer 3, and the number of neurons is set to 256, 512, and 768, respectively. Detailed parameters are shown in Table 4.

Table 4. The number of neurons and units of each of the neural networks.

Layers	Neurons	Units
1	256	256
	512	512
	768	768
3	256	64 + 64 + 128
	512	128 + 128 + 256
	768	256 + 256 + 256

The various architectures of the neural network are shown in Figure 3. Part of the figure only shows one layer of the architecture of each deep learning network. But, we actually conducted experiments using one- and three-layer stacking architectures. At the output layer, it is worth noting that we will use excitation functions for the classification tasks, binary classification will use Sigmoid, and multivariate classification will use Soft-max. We will describe the detailed parameter quantities of each neural network in the following sections.

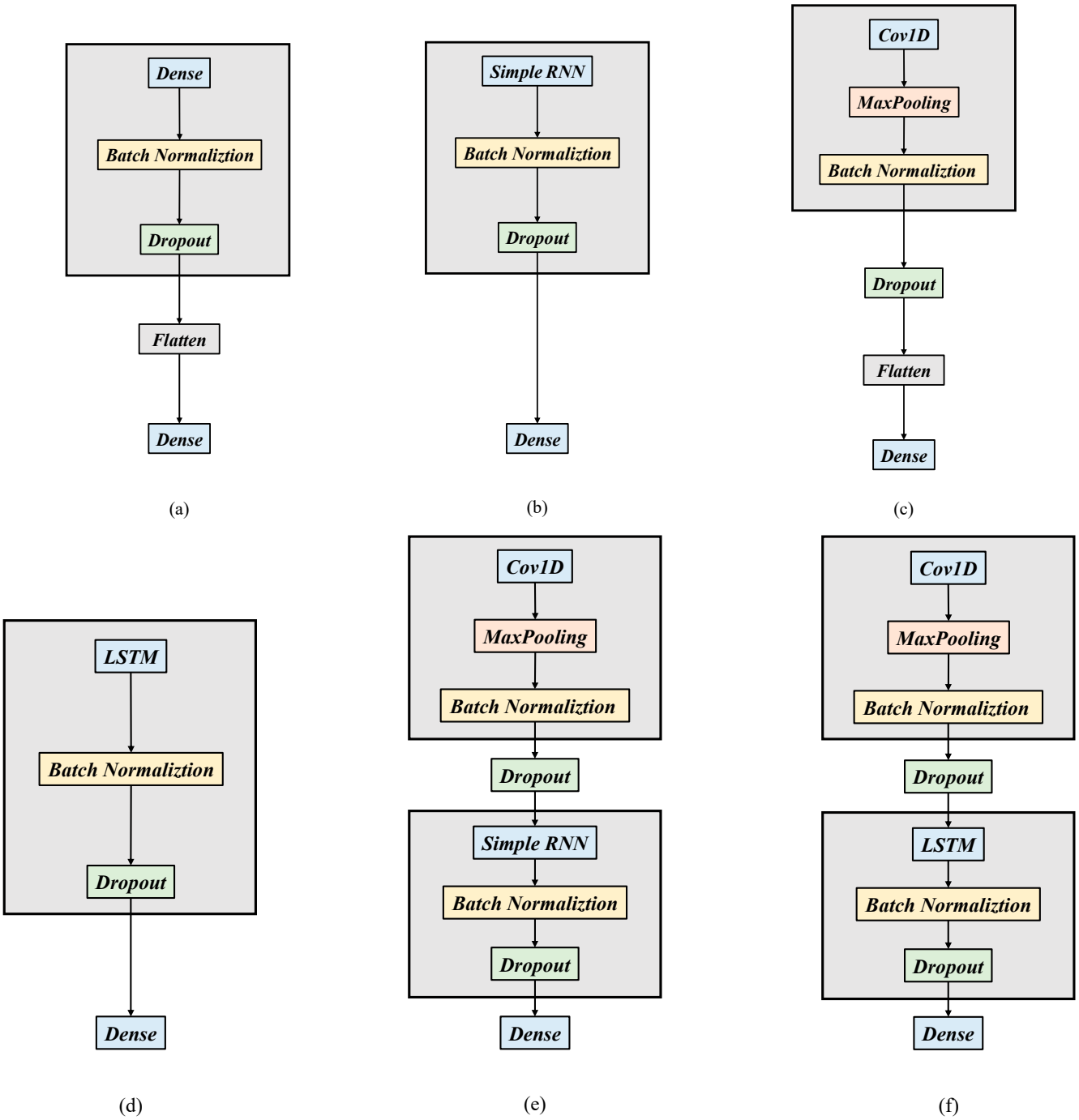


Figure 3. (**a**) Architecture diagram of DNN, (**b**) architecture diagram of RNN, (**c**) architecture diagram of CNN, (**d**) architecture diagram of LSTM, (**e**) architecture diagram of CNN + RNN, and (**f**) architecture diagram of CNN + LSTM.

3.1.1. DNN

The architecture of DNN is shown in Figure 3a, which mainly consists of the input Dense layer, Batch Normalization (BN) layer, Dropout layer, Flatten layer, and output Dense layer. The number of parameters in each layer and the corresponding number of nodes are shown in Table 5. In order to reduce the occurrence of overfitting, we add a BN layer and a Dropout layer to each layer, normalize each batch during the training process, and the Dropout layer randomly discards neurons at a certain proportion in each layer. Both effectively prevent neurons from becoming overly dependent on certain features.

Table 5. Number of parameters and nodes of DNN.

Layers	Neurons	Parameters	
		Binary	Multi-Class
1	256	13,313	15,112
	512	26,625	30,216
	768	39,937	45,320
3	256	19,521	19,976
	512	63,617	64,520
	768	146,945	148,744

3.1.2. RNN

The architecture of RNN is shown in Figure 3b. Similar to DNN, it also consists of a Simple RNN, BN layer, and Dropout layer. But, there is no Flatten layer in RNN. This is because, in RNN, the input can be a sequence, such as a text sentence or a time series, and the RNN layer is designed to be able to process sequence data. Therefore, there is no need to add a Flatten layer to convert the dimensions of the data. The number of parameters in each layer and the corresponding number of nodes are shown in Table 6.

Table 6. Number of parameters and nodes of RNN.

Layers	Neurons	Parameters	
		Binary	Multi-Class
1	256	78,849	80,648
	512	288,769	292,360
	768	629,761	635,144
3	256	44,097	44,552
	512	161,921	162,824
	768	343,553	345,352

3.1.3. CNN

The architecture of CNN is shown in Figure 3c, which mainly consists of Conv1D and MaxPooling layers. Unlike DNN and RNN where each hidden layer contains a BN layer and Dropout layer, CNN only introduces a BN layer and Dropout layer before the output layer. This design choice is attributed to the effectiveness of MaxPooling layers 1 and 2 in preventing overfitting. These layers facilitate feature extraction after convolution, emphasizing key data and minimizing irrelevant noise. Table 7 outlines the details of the number of parameters per layer and the corresponding number of nodes of CNN.

Table 7. Number of parameters and nodes of CNN.

Layers	Neurons	Parameters	
		Binary	Multi-Class
1	256	13,313	15,112
	512	26,625	30,216
	768	39,937	45,320
3	256	19,521	19,976
	512	63,617	64,520
	768	146,945	148,744

3.1.4. LSTM

The architecture of LSTM is shown in Figure 3d. LSTM is a variant of RNN designed to better handle long sequence dependencies and overcome the vanishing gradient problem of traditional RNN. The number of parameters in each layer and the corresponding number

of nodes are shown in Table 7. The architecture of CNN + RNN is shown in Figure 3e. In this architecture, there are two architectures: one with one convolutional layer and one recurrent layer, and one with three convolutional layers and three recurrent layers. The number of parameters in each layer and the corresponding number of nodes are shown in Table 8.

Table 8. Number of parameters and nodes of LSTM.

Layers	Neurons	Parameters	
		Binary	Multi-Class
1	256	311,553	313,352
	512	1,147,393	1,150,984
	768	2,507,521	2,512,904
3	256	173,121	173,576
	512	354,433	619,528
	768	1,364,225	1,366,024

3.1.5. CNN + RNN

The architecture of CNN + RNN is shown in Figure 3e. In this architecture, there are two architectures: one with one convolutional layer and one recurrent layer, and one with three convolutional layers and three recurrent layers. The number of parameters in each layer and the corresponding number of nodes are shown in Table 9.

Table 9. Number of parameters and nodes of CNN + RNN.

Layers	Neurons	Parameters	
		Binary	Multi-Class
1	256	78,849	133,160
	512	288,769	365,864
	768	629,761	729,640
3	256	44,097	86,568
	512	161,921	215,336
	768	343,553	397,864

3.1.6. CNN + LSTM

The architecture of CNN + RNN is shown in Figure 3f. In this architecture, there are two architectures: one with one convolutional layer and one recurrent layer, and one with three convolutional layers and three recurrent layers. The number of parameters in each layer and the corresponding number of nodes are shown in Table 10.

Table 10. Number of parameters and nodes of CNN + LSTM.

Layers	Neurons	Parameters	
		Binary	Multi-Class
1	256	420,041	428,840
	512	1,346,849	1,350,440
	768	2,790,945	2,796,328
3	256	246,625	247,080
	512	756,641	757,544
	768	1,479,713	1,481,512

3.2. Transformer

The architecture of the Transformer used in this paper is shown in Figure 4, and the detailed parameters are shown in Table 11. The main architecture of Transformer includes

an encoder and a decoder, but for binary and multivariate classification tasks involving a single output sequence, the decoder is unnecessary. Therefore, only encoders [7] are used in our architecture.

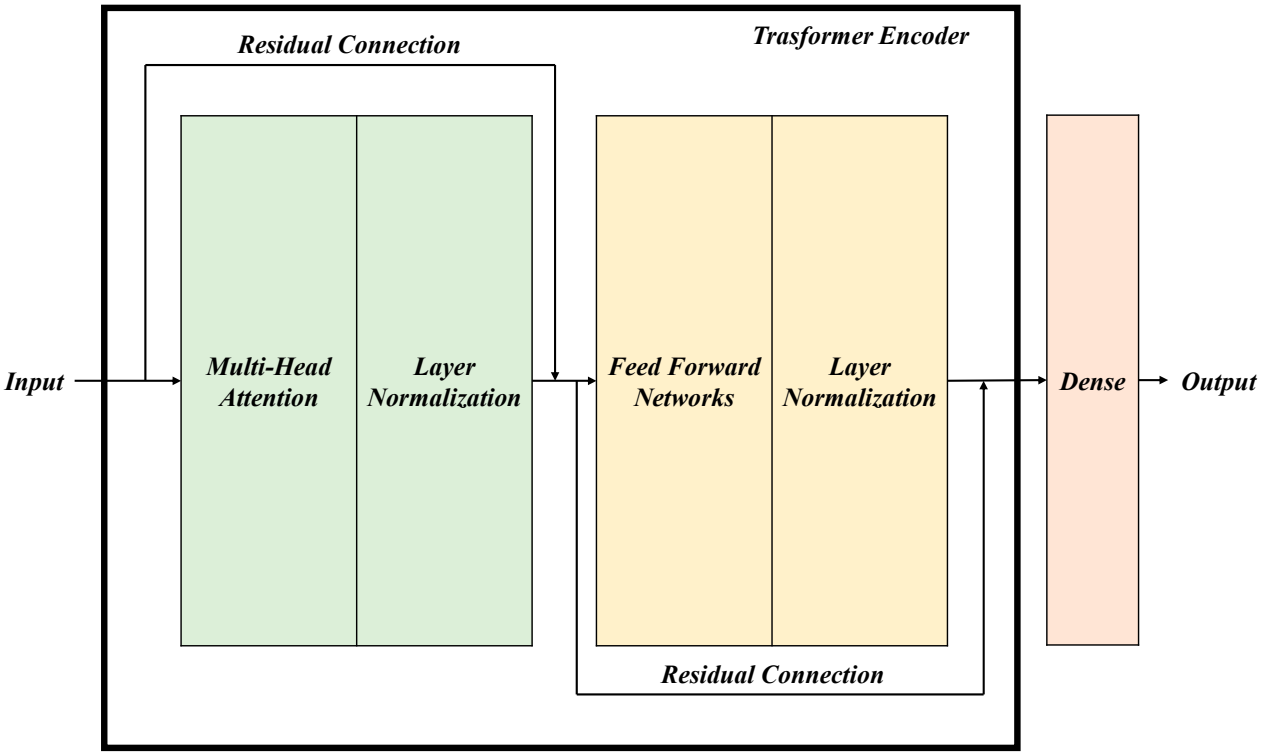


Figure 4. Transformer encoder architecture diagram.

Table 11. Number of parameters of Transformer.

Dense Dimension (FFN)	Number of Heads	Number of Layers (Encoder)	Parameters	
			Binary	Multi-Class
256	1	1	32,733	33,062
128			20,829	21,158
512			56,541	56,870
1024			104,157	104,486
2048			199,389	199,718
	2		41,335	41,664
	4		58,539	58,868
	8		94,947	93,276
		2	41,381	41,710
		4	58,677	59,006
		8	94,269	93,598

Additionally, two structures can be omitted for classification purposes. First, word embedding, which converts language vocabulary into a vector space for deep learning analysis, is unnecessary for our model. The material we are classifying is already in numeric form and converted to integers, thus eliminating the need for word embeddings. Secondly, positional encoding (Positional Encoding) used to determine the relative and absolute positions of tokens in sentences is not needed for our dataset. The length and composition of similar "sentences" in our data are fixed, making this structure not necessary [5].

3.2.1. Self Attention

The most important structures in Transformer are the self-attention mechanism and the multi-head attention mechanism. The schematic diagram of finding one of the outputs b_1 is shown in Figure 5.

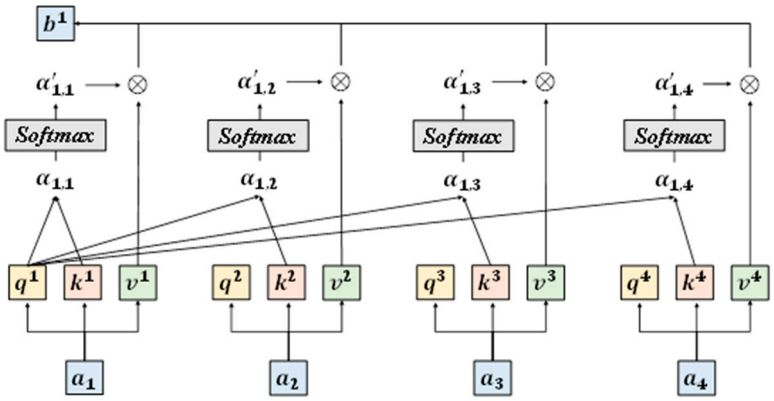


Figure 5. The schematic diagram of finding one of the outputs b_1.

First, we assume that the input is a sequence of four vectors a_1, a_2, a_3, a_4, and then multiply these four vectors by three transformation matrices W^Q, W^K and W^V to get each q^i, k^i and v^i corresponding to each input vector, that is:

$$q^i = W^Q a^i \tag{1}$$

$$k^i = W^k a^i \tag{2}$$

$$v^i = W^v a^i \tag{3}$$

where $i = 1, 2, 3, 4$.

After getting these three elements, we can start attention, as shown in Figure 5. Here, we take the output b_1 as an example.

First, we perform Scaled Dot Product on q^1 with k^1, k^2, k^3 and k^4, and we can get $\alpha_{1,1}$, $\alpha_{1,2}$, $\alpha_{1,3}$ and $\alpha_{1,4}$.

Then, we perform Softmax on $\alpha_{1,1}$, $\alpha_{1,2}$, $\alpha_{1,3}$ and $\alpha_{1,4}$, we can get $\alpha'_{1,1}$, $\alpha'_{1,2}$, $\alpha'_{1,3}$ and $\alpha'_{1,4}$, and then $\alpha'_{1,1}$, $\alpha'_{1,2}$.

$$\alpha_{1,1} = q^1 \cdot k^1 \tag{4}$$

$$\alpha_{1,2} = q^1 \cdot k^2 \tag{5}$$

$$\alpha_{1,3} = q^1 \cdot k^3 \tag{6}$$

$$\alpha_{1,4} = q^1 \cdot k^4 \tag{7}$$

$\alpha'_{1,3}$ and $\alpha'_{1,4}$ are multiplied by v^1, v^2, v^3 and v^4, respectively, and finally the four results are added to obtain the output b_1, that is:

$$b_1 = \sum_{i=1}^{4} \alpha'_{1,i} v^i = \sum_{i=1}^{4} Softmax(\alpha_{1,i}) v^i \tag{8}$$

As for b_2, b_3 and b_4, we can refer to Formula (8) and express it as the following formula:

$$b_2 = \sum_{i=1}^{4} \alpha'_{2,i} v^i \tag{9}$$

$$b_3 = \sum_{i=1}^{4} \alpha'_{3,i} v^i \tag{10}$$

$$b_4 = \sum_{i=1}^{4} \alpha'_{4,i} v^i \tag{11}$$

3.2.2. Multi-Head Attention

There is an advanced version of self-attention called the multi-head attention mechanism. In the previous chapter, the input was multiplied only once by the transformation matrices W^Q, W^K, and W^v, and then its corresponding q , k , and v values.

In the multi-head attention mechanism, taking two inputs a_1 and a_2 as an example, q, k, and v will be multiplied again by a transformation matrix. Assuming there are two attention heads, two types of q, k, and v will be obtained, respectively. As shown in Figure 6a, the first attention head $q^{1,1}$ will perform an attention calculation with $k^{1,1}$, then it will perform Softmax, and then it will multiply by $v^{1,1}$. Next, $q^{1,1}$ will be calculated with $k^{2,1}$ for attention, then Softmax, and finally multiplied by $v^{2,1}$. Finally, adding the previous two results gives $b^{1,1}$, that is:

$$b^{1,1} = \sum_{i=1}^{n} Softmax\left(q^{1,1} \cdot k^{n,1}\right) v^{n,1} \tag{12}$$

where n is 2, which is the number of heads.

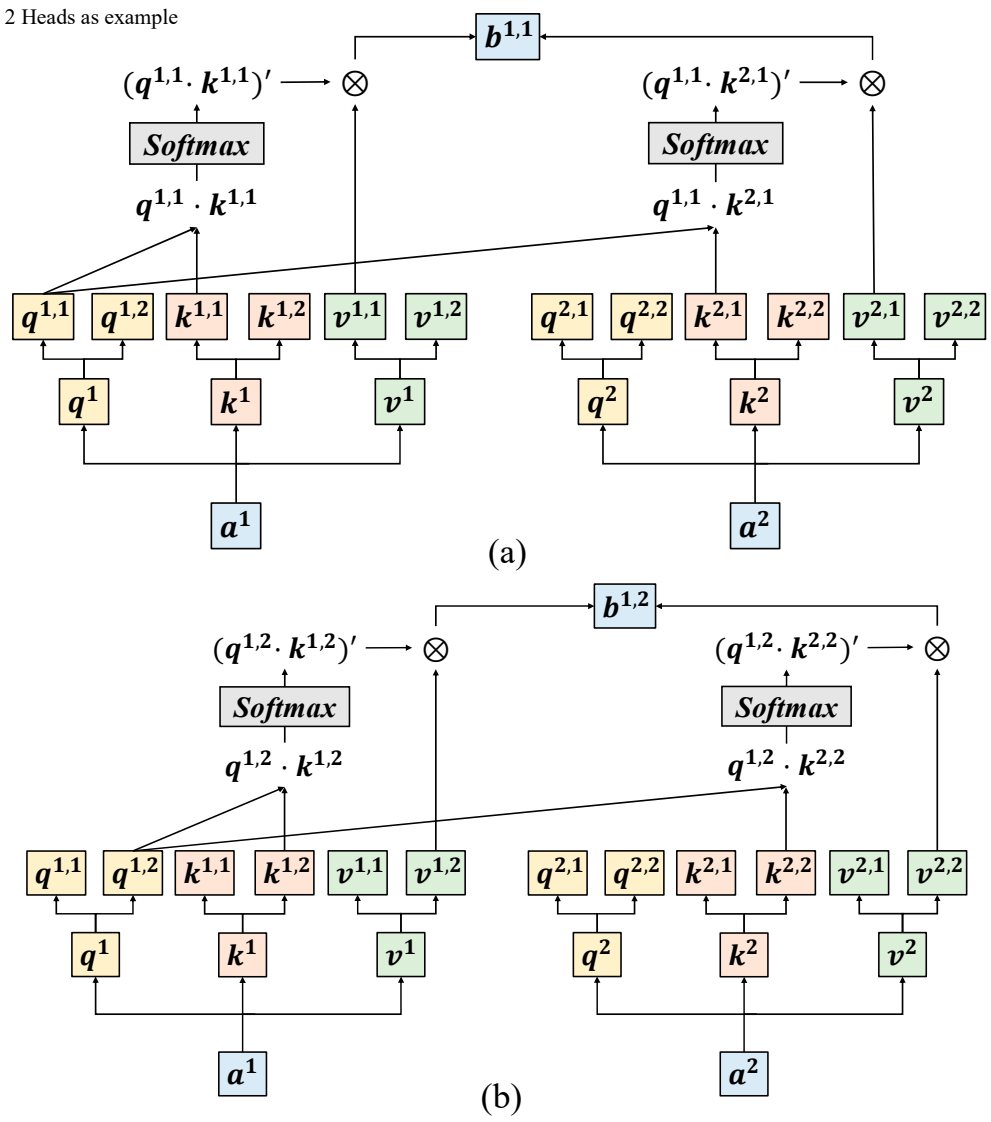
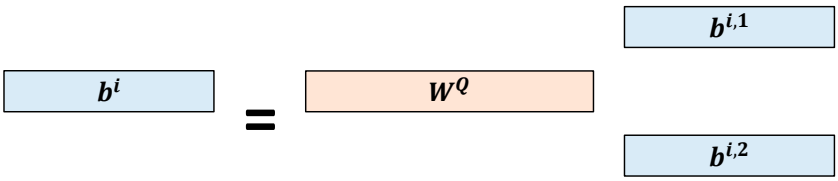


Figure 6. *Cont.*



(c)

Figure 6. (**a**) The schematic diagram of finding one of the output $b^{1,1}$; (**b**) the schematic diagram of finding one of the output $b^{1,2}$; and (**c**) the schematic diagram of adding two results.

Then, as shown in Figure 6b, the second attention head $q^{1,2}$ will perform an attention calculation with $k^{1,2}$, then it will perform Softmax, and finally it will multiply by $v^{1,2}$. Then, $q^{1,2}$ performs an attention calculation with $k^{2,2}$, then it performs Softmax, and finally it multiplies $v^{2,2}$. Finally, adding the previous two results gives $b^{1,2}$, that is:

$$b^{1,2} = \sum_{i=1}^{n} Softmax\left(q^{1,2} \cdot k^{n,1}\right) v^{n,1} \tag{13}$$

Finally, these two outputs are concatenated and multiplied by an output transformation matrix W^O to obtain the final output b^1, as shown in Figure 6c.

3.2.3. Feed Forward Network

In our architecture, the main classification task is performed in a feed forward network. The feed forward network lies behind the multi-head attention mechanism and consists of two fully connected layers. The activation function of the first layer is Relu, and no activation function is used in the second layer.

3.2.4. Layer Normalization

Layer Normalization is a technique that normalizes each input feature independently, aiming to eliminate scale differences between different features and maintain output stability. Layer normalization helps control the output of each layer to keep it within a smaller range, helping to prevent gradient explosion. Sometimes, it can accelerate the convergence of the model and improve the training speed. Compared with Batch Normalization, Layer Normalization does not need to consider batch information.

3.2.5. Residual Connection

In neural networks, complex features are learned by stacking multiple layers. However, as the number of network layers increases, the gradient may gradually decrease, making the training process difficult. The idea of residual connections is to introduce skip connections, allowing the network to directly skip one or more layers and add the input signal to the output signal. In this way, even in deep networks, the information of the original input signal can still be propagated directly to deeper layers, thus helping to alleviate the vanishing gradient problem.

4. Experimental Results

4.1. Experimental Environment

The equipment specifications and environment settings used in this article are shown in Table 12. Since simply using tensorflow will cause the training speed to be too slow; this article chooses to use tensorflow-gpu to run our model to speed up the training. The

hyperparameters of the six neural network models are shown in Table 13. Due to the large size of the dataset, we increased the batch size to 1024.

Table 12. Number of parameters of Transformer.

Project	Properties
OS	Windows 11
CPU	Intel® Core™ i7-13700 Processor
GPU	NVIDA Geforce RTX 4080
Memory	128 GB
Disk	1TB SSD
Python	3.7.16
NVIDIA CUDA	11.3.1
Framework	Tensorflow-gpu 2.5 & 2.6

Table 13. Number of parameters of Transformer.

Hyperparameter	Value
Batch Size	1024
Epochs	10
Learning Rate	0.001
Dropout	0.1

4.2. Experimental Metrics

We employ four metrics to evaluate the model's predictions of the number of accurate and inaccurate outcomes. These metrics are as follows: (1) True Positives (TPs), which represent the number of correctly classified benign samples; (2) False Positives (FPs), which represent the number of attack samples that are incorrectly predicted to be benign; (3) True Negatives (TNs), which represent the correct number of classified attack samples; and (4) False Negatives (FNs), indicating the number of benign samples that are incorrectly predicted as attacks. These four metrics produce four evaluation metrics: accuracy, precision, recall, and F1-Score. Accuracy measures the proportion of correctly classified samples. Precision measures the accuracy of predicting benign samples, while recall measures the accuracy of identifying benign samples. The F1-Score is an indicator of the classification model's performance and is the harmonic mean of precision and recall. The formulas for these metrics are summarized below:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{14}$$

$$Precision = \frac{TP}{TP + FP} \tag{15}$$

$$Recall = \frac{TP}{TP + FN} \tag{16}$$

$$F1 - Score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{17}$$

4.3. Experimental Result

The accuracy results of DNN are shown in Table 14, and the evaluation results of DNN are shown in Table 15.

Table 14. The accuracy results of DNN.

Layers	Neurons	Accuracy (%)	
		Binary	Multi-Class
1	256	99.48	97.35
	512	99.47	97.73
	768	99.53	99.13

Table 14. *Cont.*

Layers	Neurons	Accuracy (%)	
		Binary	Multi-Class
3	256	99.56	99.16
	512	99.56	99.23
	768	99.56	99.36

Table 15. The evaluation results of DNN.

Layer	Node	Precision (%)		Recall (%)		F1-Score (%)	
		Binary	Multi-Class	Binary	Multi-Class	Binary	Multi-Class
1	256	99.51	97.35	99.48	97.35	99.49	97.30
	512	99.51	97.74	99.48	97.73	99.49	97.66
	768	99.49	99.12	99.47	99.13	99.48	99.10
3	256	99.54	99.17	99.53	99.16	99.54	99.12
	512	99.57	99.24	99.56	99.23	99.56	99.18
	768	99.57	99.35	99.56	99.36	99.57	99.32

The accuracy results of RNN are shown in Table 16, and the evaluation results of RNN are shown in Table 17.

Table 16. The accuracy results of RNN.

Layers	Neurons	Accuracy (%)	
		Binary	Multi-Class
1	256	99.49	99.21
	512	99.49	99.22
	768	99.48	99.24
3	256	99.53	99.26
	512	99.50	99.27
	768	99.50	99.28

Table 17. The evaluation results of RNN.

Layer	Node	Precision (%)		Recall (%)		F1-Score (%)	
		Binary	Multi-Class	Binary	Multi-Class	Binary	Multi-Class
1	256	99.51	99.21	99.49	99.21	99.50	99.17
	512	99.50	99.23	99.49	99.22	99.49	99.19
	768	99.51	99.23	99.48	99.24	99.49	99.21
3	256	99.54	99.26	99.53	99.26	99.53	99.21
	512	99.50	99.27	99.50	99.27	99.50	99.24
	768	99.52	99.28	99.50	99.28	99.51	99.23

The accuracy results of CNN are shown in Table 18, and the evaluation results of CNN are shown in Table 19.

Table 18. The evaluation results of CNN.

Layer	Node	Precision (%)		Recall (%)		F1-Score (%)	
		Binary	Multi-Class	Binary	Multi-Class	Binary	Multi-Class
1	256	99.51	99.21	99.49	99.21	99.50	99.17
	512	99.50	99.23	99.49	99.22	99.49	99.19
	768	99.51	99.23	99.48	99.24	99.49	99.21
3	256	99.54	99.26	99.53	99.26	99.53	99.21
	512	99.50	99.27	99.50	99.27	99.50	99.24
	768	99.52	99.28	99.50	99.28	99.51	99.23

Table 19. The evaluation results of CNN.

Layer	Node	Precision (%)		Recall (%)		F1-Score (%)	
		Binary	Multi-Class	Binary	Multi-Class	Binary	Multi-Class
1	256	99.30	96.11	99.27	96.06	99.28	95.93
	512	99.29	97.83	99.27	97.73	99.28	97.64
	768	99.31	91.95	99.24	90.91	99.27	89.88
3	256	99.50	99.18	99.48	99.19	99.48	99.15
	512	99.51	99.21	99.48	99.23	99.49	99.1
	768	99.52	99.23	99.48	99.25	99.50	99.21

The accuracy results of LSTM are shown in Table 20, and the evaluation results of LSTMare shown in Table 21.

Table 20. The accuracy results of LSTM.

Layers	Neurons	Accuracy (%)	
		Binary	Multi-Class
1	256	99.51	99.28
	512	99.51	99.28
	768	99.50	99.28
3	256	99.54	99.32
	512	99.54	99.21
	768	99.52	99.34

Table 21. The evaluation results of LSTM.

Layer	Node	Precision (%)		Recall (%)		F1-Score (%)	
		Binary	Multi-Class	Binary	Multi-Class	Binary	Multi-Class
1	256	99.52	99.27	99.51	99.28	99.51	99.24
	512	99.53	99.28	99.51	99.28	99.52	99.25
	768	99.53	99.28	99.50	99.28	99.51	99.24
3	256	99.55	99.31	99.54	99.32	99.54	99.28
	512	99.55	99.31	99.54	99.31	99.54	99.28
	768	99.54	99.32	99.54	99.34	99.52	99.31

The accuracy results of CNN + RNN are shown in Table 22, and its evaluation results are shown in Table 23.

Table 22. The accuracy results of CNN + RNN.

Layers	Neurons	Accuracy (%)	
		Binary	Multi-Class
1	256	99.37	99.15
	512	99.29	99.19
	768	99.45	99.11
3	256	99.46	99.16
	512	99.42	99.07
	768	99.15	99.03

Table 23. The evaluation results of CNN + RNN.

Layer	Node	Precision (%)		Recall (%)		F1-Score (%)	
		Binary	Multi-Class	Binary	Multi-Class	Binary	Multi-Class
1	256	99.44	99.15	99.37	99.15	99.39	99.10
	512	99.36	99.19	99.29	99.19	99.32	99.15
	768	99.48	99.12	99.45	99.11	99.47	99.04
3	256	99.48	99.15	99.46	99.16	99.47	99.12
	512	99.43	99.07	99.42	99.07	99.43	99.00
	768	99.23	99.02	99.15	99.03	99.18	98.98

The accuracy results of CNN + LSTM are shown in Table 24, and its evaluation results are shown in Table 25.

Table 24. The accuracy results of CNN + LSTM.

Layers	Neurons	Accuracy (%)	
		Binary	Multi-Class
1	256	99.56	99.33
	512	99.46	98.70
	768	99.55	99.34
3	256	99.53	99.31
	512	99.49	99.26
	768	99.48	99.26

Table 25. The evaluation results of CNN + LSTM.

Layer	Node	Precision (%)		Recall (%)		F1-Score (%)	
		Binary	Multi-Class	Binary	Multi-Class	Binary	Multi-Class
1	256	99.57	99.31	99.56	99.33	99.56	99.30
	512	9.57	98.70	99.56	98.70	99.56	98.66
	768	99.57	99.33	99.55	99.34	99.56	99.31
3	256	99.55	99.29	99.53	99.31	99.54	99.28
	512	99.49	99.25	99.49	99.26	99.49	99.22
	768	99.48	99.25	99.48	99.26	99.48	99.22

The accuracy results of Transformer are shown in Table 26 and its evaluation results are shown in Tables 27 and 28.

Table 26. The accuracy results of Transformer.

Dense Dimension (FFN)	Number of Heads	Number of Layers (Encoder)	Accuracy (%)	
			Binary	Multi-Class
256	1	1	99.51	99.12
128			99.50	97.54
512			99.51	99.40
1024			99.51	99.36
2048			99.52	99.21
	2		99.50	99.19
	4		99.50	98.96
	8		99.51	99.32
		2	99.50	99.34
		4	99.49	99.23
		8	99.48	99.24

Table 27. The precision of Transformer.

Dense Dimension (FFN)	Number of Heads	Number of Layers (Encoder)	Binary	Multi-Class
256	1	1	99.52	94.03
128			99.53	98.72
512			99.52	99.27
1024			99.54	99.31
2048			99.54	99.33
	2		99.53	98.88
	4		99.52	99.23
	8		99.53	95.03
		2	99.53	99.25
		4	99.52	99.32
		8	99.49	99.11

Table 28. The recall of Transformer.

Dense Dimension (FFN)	Number of Heads	Number of Layers (Encoder)	Binary	Multi-Class
256	1	1	99.50	93.68
128			99.51	98.72
512			99.51	99.27
1024			99.52	99.43
2048			99.52	99.33
	2		99.50	98.88
	4		99.50	94.94
	8		99.51	98.88
		2	99.50	99.24
		4	99.49	99.30
		8	99.48	99.11

4.4. Accuracy Figure

In this subsection, we show the comparison between the validation and training accuracy in every model. In Figures 7–13, we provide the most complex case for each model (DNN, RNN, CNN, LSTM, CNN + RNN, CNN + LSTM, Transformer, etc.). As shown in Figures 7–13, there is no overfitting.

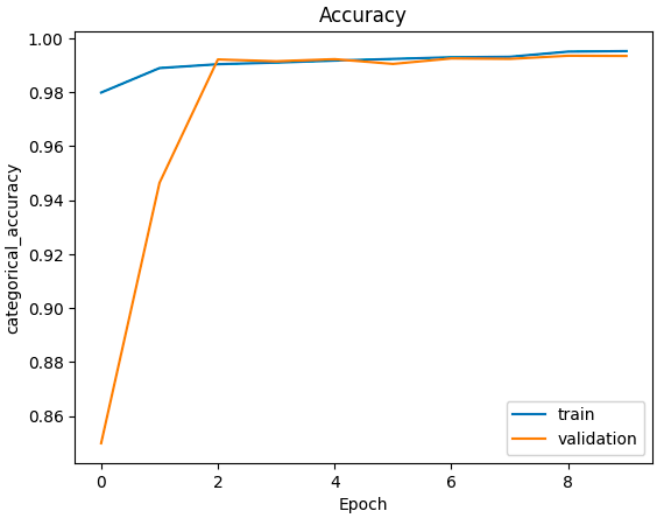


Figure 7. Accuracy figure of DNN with (layer = 3, Node = 768, multi-class).

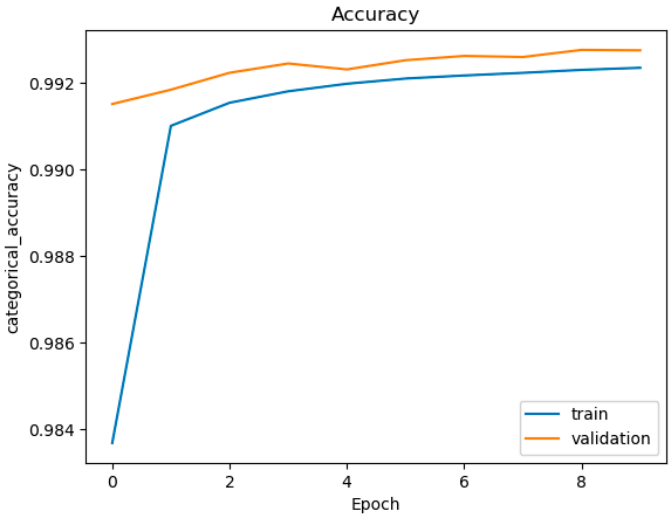


Figure 8. Accuracy figure of RNN (with layer = 3, node = 768, multi-class classification).

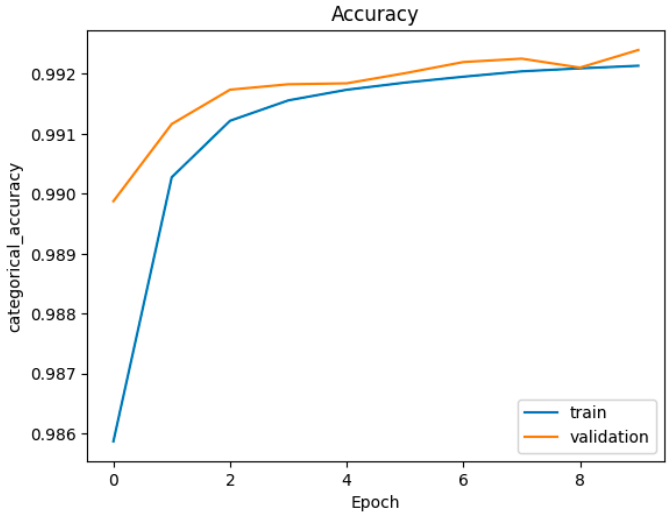


Figure 9. Accuracy figure of CNN (with layer = 3, node = 768, multi-class classification).

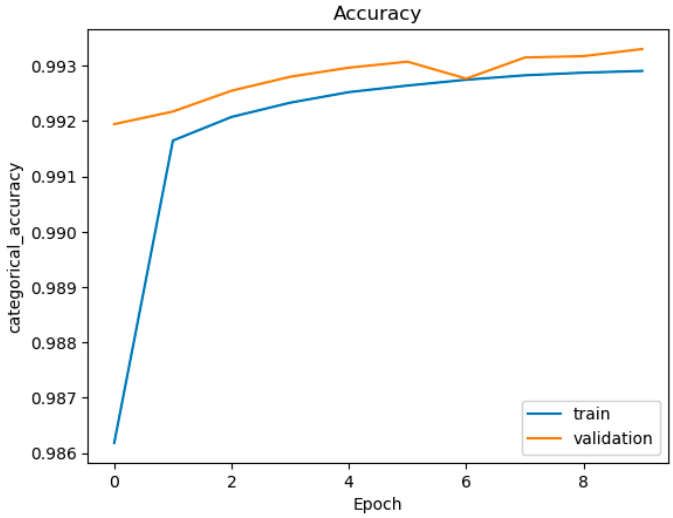


Figure 10. Accuracy figure of LSTM (with layer = 3, node = 768, multi-class classification).

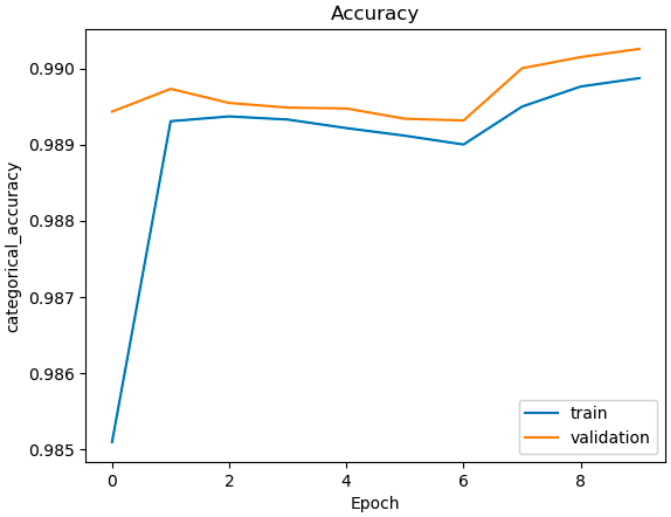


Figure 11. Accuracy figure of CNN + RNN (with layer = 3, node = 768, multi-class classification).

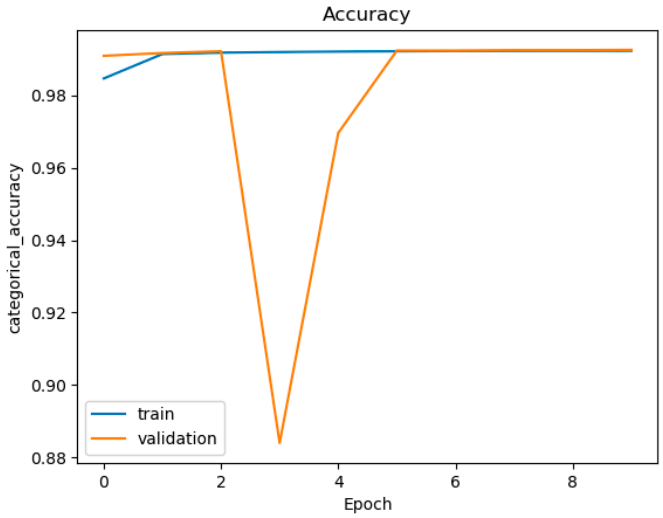


Figure 12. Accuracy figure of CNN + LSTM (with layer = 3, node = 768, multi-class classification).

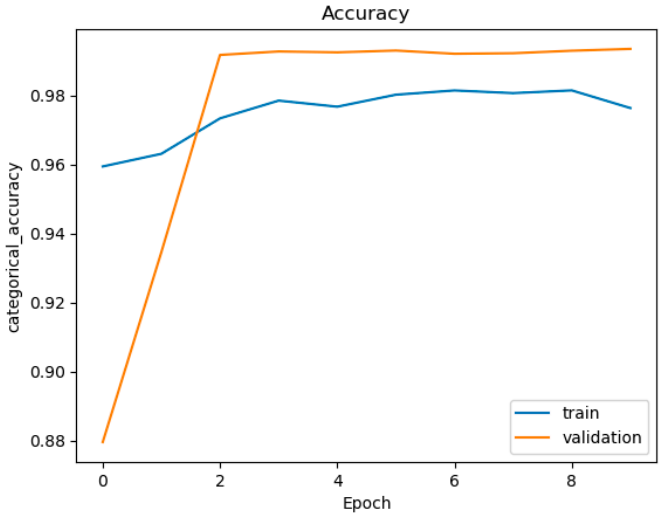


Figure 13. Accuracy figure of Transformer (with Dense Dimension = 2048, Number of Heads = 1, Number of Layers = 1, multi-class classification).

4.5. Time Consumption

The time consumption of each model is show in Table 29.

Table 29. Time consumption of each model (per sample).

Model	Binary Testing Time (μs)	Multi-Class Testing Time (μs)
DNN	3.8	3.8
RNN	7	7
CNN	12.3	12.3
LSTM	8	8
CNN + RNN	15	15
CNN + LSTM	18	18
Transformer	5	5

4.6. Confusion Matrices

In this subsection, we show the confusion matrix in every model. In Tables 30–36 we provide the most complex case for each model (DNN, RNN, CNN, LSTM, CNN + RNN, CNN + LSTM, Transformer, etc.).

Table 30. Confusion matrix figure of DNN (with layer = 3, node = 768, multi-class classification).

Actual									
	Benign Traffic	1,073,132	87	287	8001	30	3	16,647	8
	DDos	47	83,980,302	2712	1338	0	0	12	149
	Dos	22	18,808	8,071,716	79	0	0	34	7915
	Recon	82,758	5445	105	220,880	1550	138	43,664	15
	Web-Based	5367	0	7	3462	3193	12	12,787	1
	Brute Force	2508	0	2	1938	15	3749	4852	0
	Spoofing	56,557	132	141	13,208	91	945	415,405	25
	Mirai	9	13,504	289	1175	0	0	18	2,619,129
		Benign Traffic	DDos	Dos	Recon	Web-Based	Brute Force	Spoofing	Mirai

Table 31. Confusion matrix figure of RNN (with layer = 3, node = 768, multi-class classification).

Actual									
	Benign Traffic	1,057,073	7	4	17,204	40	1	23,866	0
	DDos	51	83,980,261	2463	1198	0	0	96	491
	Dos	26	7272	8,083,199	32	0	0	46	163
	Recon	83,296	1312	37	236,622	196	9	33,083	10
	Web-Based	8200	0	0	5175	2746	0	8708	0
	Brute Force	4089	0	0	3834	29	2298	2812	2
	Spoofing	108,726	24	7	24,986	220	14	352,524	3
	Mirai	18	350	56	11	0	0	33	2,633,656
		Benign Traffic	DDos	Dos	Recon	Web-Based	Brute Force	Spoofing	Mirai
Predicted									

Table 32. Confusion matrix figure of CNN (with layer = 3, node = 768, multi-class classification).

Actual									
	Benign Traffic	1,034,444	14	7	22,362	127	47	41,192	2
	DDos	83	83,979,984	3238	764	0	0	63	428
	Dos	36	6228	8,084,368	20	0	0	37	49
	Recon	78,798	2093	40	236,729	790	161	35,930	24
	Web-Based	6077	1	2	5485	2960	7	10,297	0
	Brute Force	3564	0	0	3584	78	2401	3437	0
	Spoofing	101,541	23	4	24,349	880	98	359,605	4
	Mirai	5	380	63	6	0	0	16	2,633,654
		Benign Traffic	DDos	Dos	Recon	Web-Based	Brute Force	Spoofing	Mirai
Predicted									

Table 33. Confusion matrix figure of LSTM (with layer = 3, node = 768, multi-class classification).

Actual									
	Benign Traffic	1,049,179	16	3	17,245	244	34	31,472	2
	DDos	46	83,980,598	2405	1335	2	0	47	136
	Dos	24	6531	8,084,054	28	1	0	37	63
	Recon	68,011	723	29	247,281	1212	179	37,128	2
	Web-Based	5230	1	0	4826	5520	16	9235	1
	Brute Force	3258	1	0	3384	142	2864	3415	0
	Spoofing	88,611	29	30	21,880	1797	170	373,965	22
	Mirai	11	865	38	19	0	0	25	2,633,166
		Benign Traffic	DDos	Dos	Recon	Web-Based	Brute Force	Spoofing	Mirai
Predicted									

Table 34. Confusion matrix figure of CNN + RNN (with layer = 3, node = 768, multi-class classification).

Actual	Benign Traffic	1,043,235	67	3	20,089	81	2	34,715	3
	DDos	108	83,962,688	160,078	3626	0	2	290	1768
	Dos	42	29,673	8,058,272	1521	3	0	47	1180
	Recon	95,693	4048	638	217,211	55	14	36,490	416
	Web-Based	7995	7	0	5812	1501	0	9513	1
	Brute Force	4772	1	0	3641	5	1904	2741	0
	Spoofing	131,007	95	0	27,761	203	0	327,415	23
	Mirai	29	10,576	1292	1130	0	2	161	2,620,934
		Benign Traffic	DDos	Dos	Recon	Web-Based	Brute Force	Spoofing	Mirai
		Predicted							

Table 35. Confusion matrix figure of CNN + LSTM (with layer = 3, node = 768, multi-class classification).

Actual	Benign Traffic	1,042,720	31	6	25,929	367	26	29,116	0
	DDos	33	83,980,794	2611	778	0	3	83	258
	Dos	15	6435	8,084,207	10	1	0	30	40
	Recon	66,965	1731	27	251,565	1386	155	32,689	47
	Web-Based	4273	6	1	6214	5465	10	8410	0
	Brute Force	3036	1	0	3710	177	2740	3400	0
	Spoofing	93,724	109	28	26,392	2532	77	363,638	4
	Mirai	7	368	31	70	0	0	103	2,633,545
		Benign Traffic	DDos	Dos	Recon	Web-Based	Brute Force	Spoofing	Mirai
		Predicted							

Table 36. Confusion matrix figure of Transformer (with layer = 3, node = 768, multi-class classification).

Actual	Benign Traffic	1,050,021	1264	1	23,943	61	11	22,828	66
	DDos	13	83,975,357	2031	3208	1	0	688	3262
	Dos	46	25,250	8,064,500	498	0	0	60	384
	Recon	59,531	2309	2	257,601	28	7	35,007	80
	Web-Based	5513	23	0	4960	7361	0	6971	1
	Brute Force	3300	6	0	2589	2	2318	4848	1
	Spoofing	68,286	613	0	23,988	379	333	392,815	90
	Mirai	3	7796	79	212	0	0	262	2,625,772
		Benign Traffic	DDos	Dos	Recon	Web-Based	Brute Force	Spoofing	Mirai
		Predicted							

5. Conclusions

This research is based on the CIC-IoT-2023 dataset and conducts an in-depth discussion and analysis of IoT network intrusion detection. We apply deep learning methods to improve the detection performance of abnormal behaviors and intrusions. Compared with other papers, we further use the Transformer model and further use multi-class classification. The experimental results show that in binary classification, DNN and CNN + LSTM have the highest accuracy, while in multi-class classification, the Transformer model has the highest accuracy. This proves the potential application value of deep learning methods in IoT network intrusion detection. In the future, the dataset can be reconstructed and balanced to avoid the unpredictable situation of minority category attacks, so that these 34 categories can be directly used for classification to improve the generalization ability of the model and remove some features that have no impact on model classification to improve classification efficiency.

The method used in this study brings new possibilities to the field of IoT network intrusion detection. It is hoped that the results of this study can provide a valuable reference for the development of the field of IoT security.

Author Contributions: Conceptualization, S.-M.T. and Y.-C.W.; methodology, S.-M.T. and Y.-C.W.; software and data curation, Y.-Q.W.; funding acquisition, S.-M.T. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by National Science and Technology Council, Taiwan grant number NSTC 112-2221-E-027-079-MY2.

Data Availability Statement: The data can be shared up on request. The data are not publicly available due to privacy restrictions.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Abbas, S.; Al Hejaili, A.; Sampedro, G.A.; Abisado, M.A.; Almadhor, A.M.; Shahzad, T.; Ouahada, K. A novel federated edge learning approach for detecting cyberattacks in IoT infrastructures. *IEEE Access* **2023**, *11*, 112189–112198. [CrossRef]
2. Asharf, J.; Moustafa, N.; Khurshid, H.; Debie, E.; Haider, W.; Wahab, A. A review of intrusion detection systems using machine and deep learning in Internet of Things: Challenges solutions and future directions. *Electronics* **2020**, *9*, 1177. [CrossRef]
3. Dadkhah, S.; Mahdikhani, H.; Danso, P.K.; Zohourian, A.; Truong, K.A.; Ghorbani, A.A. Towards the development of a realistic multidimensional IoT profiling dataset. In Proceedings of the 2022 19th Annual International Conference on Privacy, Security & Trust (PST), Fredericton, NB, Canada, 22–24 August 2022; pp. 1–11.
4. Talpur, A.; Gurusamy, M. Machine learning for security in vehicular networks: A comprehensive survey. *IEEE Commun. Surv. Tutor.* **2022**, *24*, 346–379. [CrossRef]
5. Li, Q.F.; Liu, Y.Q.; Niu, T.; Wang, X.M. Improved Resnet Model Based on Positive Traffic Flow for IoT Anomalous Traffic Detection. *Electronics* **2023**, *12*, 3830. [CrossRef]
6. Wang, Y.C.; Yng, Y.C.; Chen, H.X.; Tseng, S.M. Network anomaly intrusion detection based on deep learning approach. *Sensors* **2023**, *23*, 2171. [CrossRef] [PubMed]
7. Ahmed, S.W.; Kientz, F.; Kashef, R. A modified transformer neural network (MTNN) for robust intrusion detection in IoT networks. In Proceedings of the 2023 International Telecommunications Conference (ITC-Egypt), Alexandria, Egypt, 18–20 July 2023; pp. 663–668.
8. Mezina, A.; Burget, R.; Travieso-González, C.M. Network Anomaly Detection with Temporal Convolutional Network and U-Net model. *IEEE Access* **2021**, *9*, 143608–143622. [CrossRef]
9. He, M.S.; Wang, X.J.; Wei, P.; Yang, L.; Teng, Y.L.; Lyu, R.J. Reinforcement learning meets network intrusion detection: A transferable and adaptable framework for anomaly behavior identification. *IEEE Trans. Netw. Serv. Manag.* **2024**, *21*, 2477–2492. [CrossRef]
10. Jony, A.I.; Arnob, A.K.B. A long short-term memory based approach for detecting cyber attacks in IoT using CIC-IoT2023 dataset. *J. Edge Comput.* **2024**, *3*, 28–42. [CrossRef]
11. Jaradat, A.S.; Nasayreh, A.; Al-Na'amneh, Q.; Gharaibeh, H.; Al Mamlook, R.E. Genetic optimization techniques for enhancing web attacks classification in machine learning. In Proceedings of the IEEE International Conference on 11 Dependable 2023, Autonomic & Secure Computing, Abu Dhabi, United Arab Emirates, 14–17 November 2023; pp. 0130–0136.
12. Guo, G.; Pan, X.; Liu, H.; Li, F.; Pei, L.; Hu, K. An IoT intrusion detection system based on TON IoT network dataset. In Proceedings of the 2023 IEEE 13th Annual Computing and Communication Workshop and Conference (CCWC), Las Vegas, NV, USA, 8–11 March 2023; pp. 0333–0338.

13. Neto, E.C.P.; Dadkhah, S.; Ferreira, R.; Zohourian, A.; Lu, R.; Ghorbani, A.A. CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment. *Sensors* **2023**, *23*, 5941. [CrossRef]
14. Shtayat, M.M.; Hasan, M.K.; Sulaiman, R.; Islam, S.; Khan, A.U.R. An explainable ensemble deep learning approach for intrusion detection in industrial Internet of Things. *IEEE Access* **2023**, *11*, 115047–115061. [CrossRef]
15. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, L.; Polosukhin, I. Attention is all you need. In *Advances in Neural Information Processing Systems*; MIT Press: Cambridge, MA, USA, 2023; Volume 30.
16. Haque, S.; EI-Moussa, F.; Komninos, N.; Muttukrishnan, R. A systematic review of data-driven attack detection trends in IoT. *Sensors* **2023**, *23*, 7191. [CrossRef] [PubMed]
17. Le, T.T.H.; Wardhani, R.W.; Putranto, D.S.C.; Jo, U.; Kim, H. Toward enhanced attack detection and explanation in intrusion detection system-based IoT environment data. *IEEE Access* **2023**, *11*, 131661–131676. [CrossRef]